



ProxyMaze'26

The Real-Time Proxy Intelligence Challenge

“Build the watchtower we should have had a year ago.”

CHALLENGE OVERVIEW

250	+ 20	180
Core Points	Bonus Points	Passing Score

Submitted to engineering candidates across Sri Lanka.

Evaluated entirely as a **black box**. Your HTTP API is all we will ever see.

Contents

1	Where the Story Begins	2
2	The People	3
3	The Night It All Went Wrong	4
4	The Whiteboard	6
5	Why You	7
5.1	What ProxyMaze Must Do	7
6	Score Summary	8
7	Proxy Identifiers and Ground Rules	9
	Chapter 01: Proof of Life	10
	Chapter 02: The Heartbeat	10
	Chapter 03: The Memory	10
	Chapter 04: Building the Pool	11
	Chapter 05: The Watchtower	11
	Chapter 06: The Dossier	12
	Chapter 07: The Chronicle	12
	Chapter 08: The Graveyard	13
	Chapter 09: The Alert Archive	13
7.0.1	Alert object: required fields	13
	Chapter 10: The Messenger	14
7.0.2	alert.fired payload	14
7.0.3	alert.resolved payload	14
7.0.4	Delivery requirements	15
	Chapter 11: The Integration Layer	15
	Chapter 12: The Control Room	15
8	Behavioral Rules	17
8.1	Alert Lifecycle	17

PART ONE

The Torch Labs Story

1 Where the Story Begins

There is a building in Colombo, a converted warehouse on a quiet stretch of road behind the old railway yard, where the air always smells faintly of strong tea, soldering flux, and ambition. The first-floor windows face east, and on early mornings the sunlight spills flat across long wooden desks covered in laptops, notebooks, and the kind of tangled ethernet cables that say, without saying, that this is a place where things actually get built.

This is Torch Labs.

It started with four classmates and an argument over chai. The argument was simple, but it had been brewing for years: Sri Lankan engineers were brilliant, the group agreed, but the country had been quietly exporting that brilliance for a decade. Every senior, every postgraduate, every unusually sharp junior. Gone. To Singapore. To Sydney. To London. To San Francisco. The country was producing world-class engineers and shipping them overseas like cinnamon and tea.

The four of them decided that if they were going to push back against that trajectory, they would have to do it with a company.



TorchProxies

The company they built is Torch Labs. The product they chose, the problem they planted their flag in, is the unglamorous, deeply-plumbing-shaped world of internet proxy infrastructure.

*There are two flagship products. The first is **ResidentialEdge**, a fleet of genuine residential IP addresses sourced from real devices in fifteen countries: the kind of traffic that doesn't trip detection systems because, technically, it isn't suspicious traffic at all. The second is **ISPCore**, a workhorse line of dedicated ISP-grade proxies tuned for high throughput and used heavily by market intelligence and data analytics clients across Southeast Asia, East Asia, and parts of Europe.*

The company is profitable. It is also small enough that everyone knows everyone else's coffee order. There are about thirty engineers. There is no NOC team in the formal sense; there is a Slack channel called #noc that fills up at three in the morning when something unusual happens in Frankfurt or Manila or Jakarta. There is a whiteboard at the back of the second-floor office that is never erased, because everyone is afraid of erasing the wrong line.

The company is also growing. Thousands of clients. Thousands of active proxy endpoints. Hundreds of thousands of requests per minute at peak. And, until very recently, exactly one person whose job it was to know whether any of those endpoints were actually working.

2 The People

***Sachin** is the CTO. He grew up in Galle, studied computer science at the University of Wayamba, and spent four years at a distributed systems startup in Ireland before he came home with a duffel bag and an idea. His engineers describe him in two words, depending on the mood: “methodical” on the days when the system is calm, and “terrifying” on the days when it isn’t. He runs the engineering org with a soft voice and a near-religious belief in postmortems. He keeps a small, ruled notebook in his back pocket; the team has never seen what is written in it, but everyone has seen him pull it out during incidents and write a single line, very slowly, before saying anything.*

***Hirusha** is the senior infrastructure engineer. He joined Torch Labs eighteen months ago, after five years building search infrastructure for an e-commerce company in Singapore. He came back, he told Sachin during the interview, “because my parents are getting older and because Sri Lanka is going to be ready for serious infrastructure companies in about five years and I’d rather be early than late.” Sachin hired him on the spot.*

Hirusha is the engineer whose phone gets the first call when something breaks at 11:47 at night. He has a wife, a one-year-old daughter named Senuli, and a habit of eating instant noodles at his desk because he forgets to leave for dinner. He is the human safety net for thousands of proxy endpoints. He is also, increasingly, exhausted.



TorchProxies

3 The Night It All Went Wrong

The chronology of the incident, as later reconstructed for the postmortem, went like this.

Tuesday, 11:47 PM, Colombo. Hirusha was halfway through a bowl of instant noodles at his kitchen table when his phone lit up with a Slack notification. The client was a data analytics firm in Seoul that ran nightly scraping jobs against South Korean e-commerce sites. The message, translated by his phone's auto-translate, read approximately: "Your proxies are timing out. All of them. Our job starts in four hours. We need answers now."

Hirusha put down his chopsticks. He opened his laptop. He opened the "monitoring dashboard," which was generously named. It was a half-finished Python script written eight months earlier by a summer intern who had since graduated and moved to Singapore, where he was now reportedly making three times Hirusha's salary. The script was supposed to run every fifteen minutes. According to its log, it had last run at 6:02 PM, silent for almost six hours. The intern, when finally reached at 2 AM Singapore time, did not know why.

12:04 AM. Hirusha began pinging proxies manually. He opened a terminal, copied the proxy list out of a Google Sheet that the sales team also edited, and started running curl commands. By 12:30 AM, he had ninety-six confirmed dead proxies out of two hundred and twenty in the client's dedicated ResidentialEdge pool. Forty-three percent. He had no idea when the failure had started. He had no idea whether more were dying right now, while he was looking at the ones already gone. The Google Sheet told him which proxies existed; it did not tell him whether they were alive.

12:45 AM. He realized, with the slow certainty that arrives only at very late hours, that an upstream ISP serving three of their countries had quietly updated their routing tables earlier that evening. The tables were not technically wrong. They were just different. Different enough that approximately half of his residential pool could no longer establish outbound connections.

1:30 AM. He called Sachin. Sachin answered on the second ring. He didn't sound surprised. He sounded, instead, like a man who had been quietly expecting this phone call for months.

2:15 AM. The two of them, working from opposite ends of the city, swapped routing for the affected pool to a fallback ISP. By 3:08 AM, ninety-one of the ninety-six dead proxies had recovered. The remaining five were genuinely dead and would have to be retired in the morning.

3:34 AM. Hirusha closed his laptop. His daughter Senuli was awake, gurgling quietly in her crib. The sky outside the window was the colour of weak tea. He had been awake for nineteen hours.

Wednesday, 9:12 AM. The Seoul client filed a formal SLA-credit request for the missed scraping window.

Wednesday, 11:41 AM. A second client, in Jakarta, filed a related request. Their dashboards had shown elevated error rates from roughly 8 PM the previous evening, which they had assumed was a transient issue, until they read about the Korean client's situation in a private channel on a market-data forum.

Wednesday, 4:02 PM. The Seoul client cancelled their subscription. They cited not the outage

itself (outages happen, they wrote in their cancellation email) but the fact that Torch Labs had clearly not known the outage was happening. “We can tolerate failure,” the email read. “We cannot tolerate a vendor who fails silently.”



4 The Whiteboard

The team gathered on Wednesday evening in the small conference room next to the kitchen. There were six of them: Sachin, Hirusha, two backend engineers, the head of customer success, and a junior who had been hired the week before and who had spent the entire meeting looking slightly horrified.

Sachin did not say much. He listened. He let Hirusha talk. He let the customer success lead read aloud the cancellation email, twice. Then he stood up, walked over to the whiteboard at the back of the room, the one no one ever erased, and wrote, in capital letters across the top:

PROXYMAZE

He underlined it. Twice. Then, beneath it, he wrote three lines:

- *It must watch the pool continuously, without being asked.*
- *It must know when proxies fail, and which ones are about to fail.*
- *It must wake somebody up before a client emails us.*

He capped the marker. He turned around.

“We needed this six months ago,” he said. “We needed it a year ago.” He looked at Hirusha, who looked tired. He looked at the junior, who looked terrified. He looked back at the whiteboard.

“Build it now.”



5 Why You

Torch Labs can't afford to take Hirusha off live operations long enough to build ProxyMaze from scratch. They can't hire, at least not yet, a second senior infrastructure engineer who knows the system well enough to design a continuous monitoring service from a blank page. So Sachin made a different call. He went to the universities.

This document is the brief that ProxyMaze must be built from. You are the engineer Sachin is looking for. He has no idea who you are. He hasn't seen your CV. He hasn't interviewed you. He will never read your code.

What he will do is point an HTTP client at the service you build, and use it.

If your service answers the way it should, ProxyMaze becomes Hirusha's new safety net, your name goes on a shortlist that he is keeping in that small ruled notebook in his back pocket, and Torch Labs takes a step toward never having a Wednesday like the last one again.

5.1 What ProxyMaze Must Do

1. Accept runtime configuration for monitoring cadence and request timeout, and apply it immediately.
2. Accept proxy URLs and monitor them **continuously, in the background**, on its own schedule, not only when somebody asks.
3. Track every proxy as **pending**, **up**, or **down**, classified strictly from real HTTP probes.
4. Compute the pool failure rate as $\frac{\text{down}}{\text{total}}$.
5. Fire an alert the moment the failure rate reaches the threshold, and resolve that same alert when the pool recovers below the threshold.
6. Deliver **alert.fired** and **alert.resolved** webhook events to registered receivers, including under transient failures of the receiver.
7. Keep every endpoint and every webhook payload telling exactly the same story about exactly the same monitoring state.
8. Speak Slack and Discord, so the people who actually run Torch Labs (the ones with phones, not log readers) get a useful message instead of raw JSON.

6 Score Summary

Category	Points
Service bootstrap and configuration	10
Proxy pool ingestion and background monitoring	45
Single failure behavior	30
Threshold breach alerts and webhook delivery	90
Alert resolution	20
Re-breach lifecycle integrity	30
Pool operations and observability	25
Core Total	250
Slack integration (bonus)	+10
Discord integration (bonus)	+10
Maximum Score	270
Passing Score	186



7 Proxy Identifiers and Ground Rules

Proxy IDs must be **deterministic** and **externally addressable**. For a proxy URL whose path ends in a final segment, that segment is the proxy id:

```
https://proxy-provider.example/proxy/px-101
```

↓
px-101

The same ID must appear consistently across `POST /proxies`, `GET /proxies`, `GET /proxies/{id}`, `GET /proxies/{id}/history`, the `failed_proxy_ids` field on alerts, and every webhook payload that mentions a failing proxy.

All timestamps are **ISO 8601 UTC**, e.g. `2026-04-24T10:15:30Z`. All request and response bodies are **JSON** unless otherwise stated. The threshold for the pool failure rate is `0.20`.



PART TWO

The Operational Contract

Sachin slid a printed document across the table to the team, and across this document to you. Thirteen endpoints. Each one a piece of the system that Hirusha had needed at 11:47 PM and did not have. Each one will be exercised, in ways that are not announced in advance.

The chapters that follow define what ProxyMaze must do. Not how. The “how” is your job.

CHAPTER 01 GET /health

Proof of Life

Before you can monitor anything, prove that you exist.

Returns service health. **Response** 200 OK:

```
{
  "status": "ok"
}
```

CHAPTER 02 POST /config

The Heartbeat

Set the pace. Every watchman has a frequency.

Sets the runtime monitoring configuration. The values must apply to **all subsequent health checks, immediately**. **Response** 200 OK.

```
{
  "check_interval_seconds": 15,
  "request_timeout_ms": 3000
}
```

CHAPTER 03 GET /config

The Memory

Repeat back what you heard. Trust, but verify.

Returns the currently active runtime configuration. **Response** 200 OK, body matches the most recently accepted POST /config.

CHAPTER 04 POST /proxies

Building the Pool

Load the targets. The watch begins the moment you accept them.

Loads proxy URLs into the monitoring pool.

Request:

```
{
  "proxies": [
    "https://proxy-provider.example/proxy/px-101",
    "https://proxy-provider.example/proxy/px-102"
  ],
  "replace": true
}
```

Rules:

- **replace** omitted or **false**: append the provided proxies to the current pool.
- **replace: true**: clear the current pool first, then load the provided proxies.
- Newly accepted proxies start as **pending** until their first check completes. They must transition to **up** or **down** on their own, from real probes, without being asked.
- Replacing or clearing the pool must **not** delete previous alerts.
- Additional fields not listed above may appear in the request body. They must be ignored cleanly: they must not cause the request to fail.

Response 201 Created:

```
{
  "accepted": 2,
  "proxies": [
    { "id": "px-101", "url": "...", "status": "pending" },
    { "id": "px-102", "url": "...", "status": "pending" }
  ]
}
```

CHAPTER 05 GET /proxies

The Watchtower

Survey the entire pool at a glance.

Returns the live pool summary and per-proxy state. **Response 200 OK:**

```
{
  "total": 10,
  "up": 7,
  "down": 3,
  "failure_rate": 0.3,
  "proxies": [
    {
      "id": "px-101",
```

```
"url": "https://proxy-provider.example/proxy/px-101",
"status": "up",
"last_checked_at": "2026-04-24T10:15:30Z",
"consecutive_failures": 0
}
]
```

Each proxy entry must carry, at minimum: `id`, `url`, `status`, `last_checked_at`, `consecutive_failures`. The values must reflect the latest background check, not a check freshly triggered by this request.

CHAPTER 06 GET /proxies/{id}

The Dossier

Every proxy has a story. This is where you read it.

Returns details for a single proxy. Return 404 Not Found for unknown IDs. **Response 200 OK:**

```
{
  "id": "px-101",
  "url": "https://proxy-provider.example/proxy/px-101",
  "status": "up",
  "last_checked_at": "2026-04-24T10:15:30Z",
  "consecutive_failures": 0,
  "total_checks": 12,
  "uptime_percentage": 91.7,
  "history": [
    { "checked_at": "2026-04-24T10:15:30Z", "status": "up" }
  ]
}
```

The full set of required fields is the five from GET /proxies, plus `total_checks`, `uptime_percentage`, and `history`.

CHAPTER 07 GET /proxies/{id}/history

The Chronicle

The paper trail that proves what happened, and when.

Returns the check history for a single proxy. The response body must be a **JSON array**. Return 404 Not Found for unknown IDs. **Response 200 OK:**

```
[
  { "checked_at": "2026-04-24T10:15:30Z", "status": "up" },
  { "checked_at": "2026-04-24T10:16:00Z", "status": "down" }
]
```

CHAPTER 08 DELETE /proxies

The Graveyard

Clear the board. History survives the purge.

Clears the current proxy pool. **Response:** 204 No Content.

- The proxy pool becomes empty.
- Existing alerts must remain accessible through GET /alerts.
- Alert history must **not** be deleted.

CHAPTER 09 GET /alerts

The Alert Archive

Every crisis, recorded. Active and resolved, nothing forgotten.

Returns all alerts, both active and resolved. **Response** 200 OK, body is a JSON array:

```
[
  {
    "alert_id": "alert-a1b2c3",
    "status": "active",
    "failure_rate": 0.3,
    "total_proxies": 10,
    "failed_proxies": 3,
    "failed_proxy_ids": ["px-103", "px-104", "px-105"],
    "threshold": 0.2,
    "fired_at": "2026-04-24T10:20:00Z",
    "resolved_at": null,
    "message": "Proxy pool failure rate exceeded threshold"
  }
]
```

7.0.1 Alert object: required fields

- **alert_id**: non-empty, stable for the lifetime of the alert. A new **alert_id** is minted on every fresh breach.
- **status**: "active" while the breach holds, "resolved" after recovery.
- **failure_rate**: the rate that justified the alert (≥ 0.20).
- **total_proxies**: the size of the pool at fire time.
- **failed_proxies**: count of proxies currently down.
- **failed_proxy_ids**: IDs of the proxies currently down.
- **threshold**: 0.2.
- **fired_at**: ISO 8601 UTC, the moment the breach began.
- **resolved_at**: ISO 8601 UTC once resolved, otherwise null.

- **message**: a short, non-empty human-readable summary.

Alert lifecycle. An alert is a lifecycle, not a counter. At most one alert is **active** at any time. While the breach continues, that single alert and its single **alert_id** must persist: no second active alert may appear, no matter how long the breach lasts. After resolution, a fresh breach mints a brand-new **alert_id**; the previously resolved alert remains in the archive unchanged.

CHAPTER 10 POST /webhooks

The Messenger

When the alarm fires, somebody gets woken up.

Registers a URL to receive alert webhook notifications. The request body is a JSON object. The **url** field is required; **additional fields may be present and must be accepted (and ignored) without error.**

Request example:

```
{
  "url": "https://receiver.example/proxywatch-webhook"
}
```

Response 201 Created:

```
{
  "webhook_id": "wh-123",
  "url": "https://receiver.example/proxywatch-webhook"
}
```

7.0.2 alert.fired payload

```
{
  "event": "alert.fired",
  "alert_id": "alert-a1b2c3",
  "fired_at": "2026-04-24T10:20:00Z",
  "failure_rate": 0.3,
  "total_proxies": 10,
  "failed_proxies": 3,
  "failed_proxy_ids": ["px-103", "px-104", "px-105"],
  "threshold": 0.2,
  "message": "Proxy pool failure rate exceeded threshold"
}
```

7.0.3 alert.resolved payload

```
{
  "event": "alert.resolved",
  "alert_id": "alert-a1b2c3",
  "resolved_at": "2026-04-24T10:30:00Z"
}
```

7.0.4 Delivery requirements

- Send each event with header **Content-Type:** `application/json`.
- Deliver each event to every registered receiver within **60 seconds** of the underlying state transition.
- If the receiver responds with a transient failure (500, 502, 503, or 504), retry until the delivery succeeds.
- For each state transition, exactly one *successful* delivery must reach the receiver. No duplicates, even if the breach persists for an extended period.

CHAPTER 11 `POST /integrations`

The Integration Layer

Speak the language of the team. Slack for ops. Discord for engineering.

Registers a Slack or Discord formatted alert integration. **Response:** 200 OK or 201 Created. Detailed payload requirements appear in **Part Four: Bonus Integrations**.

Slack request:

```
{
  "type": "slack",
  "webhook_url": "https://receiver.example/slack",
  "username": "ProxyWatch",
  "events": ["alert.fired", "alert.resolved"]
}
```

Discord request:

```
{
  "type": "discord",
  "webhook_url": "https://receiver.example/discord",
  "username": "ProxyWatch",
  "events": ["alert.fired", "alert.resolved"]
}
```

CHAPTER 12 `GET /metrics`

The Control Room

What gets measured, gets managed.

Returns operational monitoring data. The response body must be valid, **non-empty** JSON. **Response** 200 OK:

```
{
  "total_checks": 120,
  "current_pool_size": 10,
  "active_alerts": 1,
  "total_alerts": 3,
  "webhook_deliveries": 4
}
```

PART THREE

The Laws of ProxyMaze

Every system has rules. These are not suggestions. They are the behavioural constraints that determine whether ProxyMaze actually solves the problem Sachin wrote on the whiteboard.

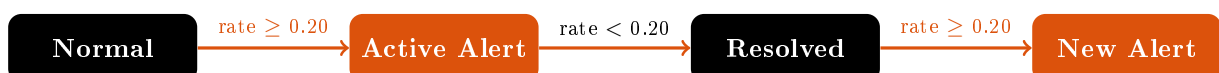


TorchProxies

8 Behavioral Rules

- ▶ Monitoring runs **continuously, in the background**, on the cadence set by `check_interval_seconds`. A service that only checks proxies when a read endpoint is called does not satisfy the requirement.
- ▶ Proxy status is derived from **real HTTP probes** to the submitted proxy URLs, never from mocked, hardcoded, or cached data.
- ▶ A 2xx response received within `request_timeout_ms` $\Rightarrow$ **up**.
- ▶ A timeout, connection failure, connection refusal, or any 5xx response $\Rightarrow$ **down**. *Both* timeout-style failures and 5xx-style failures must classify the proxy as down; both must appear in `failed_proxy_ids` during a breach.
- ▶ The alert threshold is 0.20. The alert fires when the failure rate is ≥ 0.20 and resolves when the failure rate drops below 0.20.
- ▶ Only one alert is **active** at a time. A continuous breach must not produce duplicate active alerts and must not produce duplicate successful `alert.fired` deliveries to a registered receiver.
- ▶ After an alert resolves, a fresh breach creates a **new alert** with a brand-new `alert_id`. The previously resolved alert stays in the archive.
- ▶ For each transition, the webhook events must be observable in order: `alert.fired` for the prior alert, then `alert.resolved` for the prior alert, then `alert.fired` for the new alert.
- ▶ `failed_proxy_ids` must always equal the set of proxies **currently** classified as down.
- ▶ GET `/proxies`, GET `/alerts`, and any webhook payload describing an active breach must agree exactly on the failed proxy set, on `total_proxies`, on `failed_proxies`, and on `threshold`.
- ▶ All request bodies that contain a JSON object must accept unknown fields without error. Reject only on genuinely malformed input.
- ▶ Sample proxy URLs, IDs, orderings, timings, and failure modes used during evaluation may differ from any examples in this document. The contract is what matters, not the specific examples.

8.1 Alert Lifecycle



PART FOUR

Bonus Integrations

ProxyMaze's core works without these. But the people who actually run Torch Labs (Hirusha, the customer success team, the engineers in the #noc channel) live in Slack and Discord. A system that speaks their language gets used. One that doesn't gets ignored, no matter how well it works underneath.

BONUS INTEGRATION 01

Slack | +10 pts

SLACK FORMATTED ALERTS

On every `alert.fired` (and `alert.resolved`) event, POST a Slack-style payload to the registered `webhook_url`, with `Content-Type: application/json`. The payload must contain:

- `username`: non-empty string.
- `text`: non-empty string summarising the event.
- `attachments[0].color`: a hex string of the form "#RRGGBB".
- `attachments[0].fields`: array of `{title, value}` entries. The `titles` collectively must include (case-insensitive substring match): *Alert ID*, *Failure Rate*, *Failed Proxies*, *Threshold*, *Failed IDs*, and *Fired At*.
- `attachments[0].footer`: non-empty string.
- `attachments[0].ts`: a Unix epoch timestamp expressed as an **integer** (number of whole seconds; not a float and not a string).

The Slack payload must be delivered to the registered Slack webhook URL within 60 seconds of the underlying alert event.

BONUS INTEGRATION 02

Discord | +10 pts

DISCORD FORMATTED ALERTS

On every alert event, POST a Discord-style payload to the registered `webhook_url`, with `Content-Type: application/json`. The payload must contain:

- `embeds[0].title`: non-empty string.
- `embeds[0].description`: non-empty string summarising the event.
- `embeds[0].color`: an integer in the range 0 to 16777215 inclusive.
- `embeds[0].fields`: array of `{name, value}` entries. The `names` collectively must include (case-insensitive substring match): *Alert ID*, *Failure Rate*, *Failed Proxies*, *Threshold*, and *Failed IDs*.
- `embeds[0].footer.text`: non-empty string.

The Discord payload must be delivered to the registered Discord webhook URL within 60 seconds of the underlying alert event.



TorchProxies

Good luck.

Hirusha is watching the dashboard.

Sachin is watching the scoreboard.

Sri Lanka is watching you.

Torch Labs • Colombo, Sri Lanka • *From Sri Lanka, to the world.*